

Table of Contents

- 1 Overview of Machine Learning
- 2 Convex Optimization
- 3 Singular Value Decomposition
- 4 Fourier Analysis
- 5 Linear Regression**
- 6 Dynamical Systems

House Price Prediction

You need to determine a fair market price for a house. This price depends on its characteristics:

House Price Prediction

You need to determine a fair market price for a house. This price depends on its characteristics:

- Size
- Age
- Number of Rooms
- Locations
- ...

House Price Prediction

You need to determine a fair market price for a house. This price depends on its characteristics:

- Size
- Age
- Number of Rooms
- Locations
- ...

The goal is to create a reliable system that can take these features for any house and produce an accurate price estimate.

House Price Prediction

Linear regression assumes the price is a weighted sum of its features.

Conceptually, the formula is:

$$\text{Price} = \text{Base Value} + (\text{Value per sq. ft.} \times \text{Size}) - (\text{Depreciation per year} \times \text{Age}) + (\text{Value per room} \times \text{Number of Rooms}) + \dots$$

House Price Prediction

Linear regression assumes the price is a weighted sum of its features.

Conceptually, the formula is:

Price = Base Value + (Value per sq. ft. $\times$ Size) - (Depreciation per year $\times$ Age) + (Value per room $\times$ Number of Rooms) + ...

When dealing with thousands of houses, the most efficient way to represent this entire system is with the matrix equation:

$$y = X\beta + \epsilon$$

House Price Prediction

$$y = X\beta + \epsilon$$

- y : This is the price vector. It's a single column listing the known selling price for every house in your dataset. It's the “ground truth” the model learns from.

$$y = \begin{bmatrix} \text{Price}_1 \\ \text{Price}_2 \\ \vdots \\ \text{Price}_m \end{bmatrix}$$

House Price Prediction

$$y = X\beta + \epsilon$$

- X : This is the design matrix. Each row represents a single house, and each column represents a feature. A column of ones is included to account for the model's base value (the intercept).

$$X = \begin{bmatrix} 1 & \text{Size}_1 & \text{Age}_1 & \cdots & n\text{-th Feature}_1 \\ 1 & \text{Size}_2 & \text{Age}_2 & \cdots & n\text{-th Feature}_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & \text{Size}_m & \text{Age}_m & \cdots & n\text{-th Feature}_m \end{bmatrix}$$

House Price Prediction

$$y = X\beta + \epsilon$$

- β : This is the coefficient vector. This is what the model solves for. It contains the “rules” of the market—the specific monetary value associated with each feature (e.g., the value of an extra bedroom or the depreciation from another year of age).

$$\beta = \begin{bmatrix} \text{Base Value} \\ \text{Value per sq. ft.} \\ \text{Depreciation per year} \\ \vdots \end{bmatrix}$$

House Price Prediction

$$y = X\beta + \epsilon$$

- ϵ : This is the error vector. It's a list of the differences between the actual prices (y) and the prices predicted by the model ($X\beta$). No model is perfect, and this captures the unexplained part.

House Price Prediction

$$y = X\beta + \epsilon$$

- ϵ : This is the error vector. It's a list of the differences between the actual prices (y) and the prices predicted by the model ($X\beta$). No model is perfect, and this captures the unexplained part.

By solving this equation, regression finds the optimal coefficient vector (β) that makes the total prediction error as small as possible. You're then left with a powerful model that can price any house for which you have the features.

Linear Regression

For a dataset with m samples and $n - 1$ features, we can express the entire system of linear equations efficiently using matrix and vector notation.

$$y = X\beta$$

Linear Regression

For a dataset with m samples and $n - 1$ features, we can express the entire system of linear equations efficiently using matrix and vector notation.

$$y = X\beta$$

- $y \in \mathbb{R}^{m \times 1}$: true values.
- $X \in \mathbb{R}^{m \times n}$: design matrix.
- $\beta \in \mathbb{R}^{n \times 1}$: coefficient vector.

Linear Regression

For a dataset with m samples and $n - 1$ features, we can express the entire system of linear equations efficiently using matrix and vector notation.

$$y = X\beta$$

Optimization Problem:

$$\min_{\beta} \|y - X\beta\|_2^2$$

- $y \in \mathbb{R}^{m \times 1}$: true values.
- $X \in \mathbb{R}^{m \times n}$: design matrix.
- $\beta \in \mathbb{R}^{n \times 1}$: coefficient vector.

Overdetermined vs Underdetermined Systems

- Overdetermined System ($m > n$)

Overdetermined vs Underdetermined Systems

- Overdetermined System ($m > n$)
 - More samples than features. This is the standard case.
 - More equations than unknowns. No exact solution exists that satisfies all equations perfectly.

Overdetermined vs Underdetermined Systems

- Overdetermined System ($m > n$)
 - More samples than features. This is the standard case.
 - More equations than unknowns. No exact solution exists that satisfies all equations perfectly.
 - Approximate solution: $\min_{\beta} \|y - X\beta\|_2^2$
 - Linear regression with lots of data points. You have many data points (equations) but are trying to find the single best line.

Overdetermined vs Underdetermined Systems

- Overdetermined System ($m > n$)
 - More samples than features. This is the standard case.
 - More equations than unknowns. No exact solution exists that satisfies all equations perfectly.
 - Approximate solution: $\min_{\beta} \|y - X\beta\|_2^2$
 - Linear regression with lots of data points. You have many data points (equations) but are trying to find the single best line.
- Underdetermined System ($m < n$)

Overdetermined vs Underdetermined Systems

- Overdetermined System ($m > n$)
 - More samples than features. This is the standard case.
 - More equations than unknowns. No exact solution exists that satisfies all equations perfectly.
 - Approximate solution: $\min_{\beta} \|y - X\beta\|_2^2$
 - Linear regression with lots of data points. You have many data points (equations) but are trying to find the single best line.
- Underdetermined System ($m < n$)
 - Fewer samples than features. A “high-dimensional” problem.
 - Infinite solutions.

Overdetermined vs Underdetermined Systems

- Overdetermined System ($m > n$)
 - More samples than features. This is the standard case.
 - More equations than unknowns. No exact solution exists that satisfies all equations perfectly.
 - Approximate solution: $\min_{\beta} \|y - X\beta\|_2^2$
 - Linear regression with lots of data points. You have many data points (equations) but are trying to find the single best line.
- Underdetermined System ($m < n$)
 - Fewer samples than features. A “high-dimensional” problem.
 - Infinite solutions.
 - This mathematical reality necessitates regularization to select a single, stable solution.

Solving Linear Regression

Once we have our model, $y = X\beta$, the goal is to find the coefficient vector (β) that makes the line fit our data as closely as possible. This means minimizing the error.

Solving Linear Regression

Once we have our model, $y = X\beta$, the goal is to find the coefficient vector (β) that makes the line fit our data as closely as possible. This means minimizing the error.

- Normal Equation (Lecture 2)

$$X\beta = y \Rightarrow X^T X\beta = X^T y \Rightarrow \beta = (X^T X)^{-1} X^T y$$

Solving Linear Regression

Once we have our model, $y = X\beta$, the goal is to find the coefficient vector (β) that makes the line fit our data as closely as possible. This means minimizing the error.

- Normal Equation (Lecture 2)

$$X\beta = y \Rightarrow X^T X\beta = X^T y \Rightarrow \beta = (X^T X)^{-1} X^T y$$

- **Exact Solution:** No guessing or iteration is needed. It gives you the precise answer in one calculation.
- **No Hyperparameters:** You don't need to choose a learning rate or number of iterations.

Solving Linear Regression

Once we have our model, $y = X\beta$, the goal is to find the coefficient vector (β) that makes the line fit our data as closely as possible. This means minimizing the error.

- Normal Equation (Lecture 2)

$$X\beta = y \Rightarrow X^T X\beta = X^T y \Rightarrow \beta = (X^T X)^{-1} X^T y$$

- **Exact Solution:** No guessing or iteration is needed. It gives you the precise answer in one calculation.
- **No Hyperparameters:** You don't need to choose a learning rate or number of iterations.
- **Slow for Large Datasets:** Calculating the inverse of a correlation matrix is computationally very expensive.
- **Fails if Matrix is Not Invertible:** If features are multicollinear, the correlation matrix can't be inverted, and the equation fails.

Solving Linear Regression

Once we have our model, $y = X\beta$, the goal is to find the coefficient vector (β) that makes the line fit our data as closely as possible. This means minimizing the error.

- Gradient Descent (Lecture 1)

$$\beta_{k+1} = \beta_k - \alpha \nabla f(\beta)$$

Solving Linear Regression

Once we have our model, $y = X\beta$, the goal is to find the coefficient vector (β) that makes the line fit our data as closely as possible. This means minimizing the error.

- Gradient Descent (Lecture 1)

$$\beta_{k+1} = \beta_k - \alpha \nabla f(\beta)$$

- Scales Well: Much more efficient for datasets with a very large number of features.
- Versatile: Can be used for many other machine learning algorithms, not just linear regression.

Solving Linear Regression

Once we have our model, $y = X\beta$, the goal is to find the coefficient vector (β) that makes the line fit our data as closely as possible. This means minimizing the error.

- Gradient Descent (Lecture 1)

$$\beta_{k+1} = \beta_k - \alpha \nabla f(\beta)$$

- Scales Well: Much more efficient for datasets with a very large number of features.
- Versatile: Can be used for many other machine learning algorithms, not just linear regression.
- Requires Tuning: You must choose a learning rate (how big of a step to take). If it's too large, you'll overshoot the minimum; if it's too small, it will take forever to converge.
- Approximate Solution: It finds a value very close to the optimal β , but not the exact analytical solution.
- Can be Slow: May require many iterations to converge.

Solving Linear Regression

Once we have our model, $y = X\beta$, the goal is to find the coefficient vector (β) that makes the line fit our data as closely as possible. This means minimizing the error.

- Singular Value Decomposition (Lecture 2)

$$X = U\Sigma V^T \Rightarrow y = X^+y \Rightarrow y = V\Sigma^+U^Ty$$

Solving Linear Regression

Once we have our model, $y = X\beta$, the goal is to find the coefficient vector (β) that makes the line fit our data as closely as possible. This means minimizing the error.

- Singular Value Decomposition (Lecture 2)

$$X = U\Sigma V^T \Rightarrow y = X^+y \Rightarrow y = V\Sigma^+U^Ty$$

- Always Works: Unlike the Normal Equation, SVD works even if the correlation matrix is singular.
- Provides Insights: The decomposition can reveal information about the structure of your data and the relationships between features.

Solving Linear Regression

Once we have our model, $y = X\beta$, the goal is to find the coefficient vector (β) that makes the line fit our data as closely as possible. This means minimizing the error.

- Singular Value Decomposition (Lecture 2)

$$X = U\Sigma V^T \Rightarrow y = X^+y \Rightarrow y = V\Sigma^+U^Ty$$

- Always Works: Unlike the Normal Equation, SVD works even if the correlation matrix is singular.
- Provides Insights: The decomposition can reveal information about the structure of your data and the relationships between features.
- Computationally Expensive: Like the Normal Equation, it can be slow for datasets with a huge number of features, though often more stable.

Overfitting vs Underfitting

Overfitting vs Underfitting

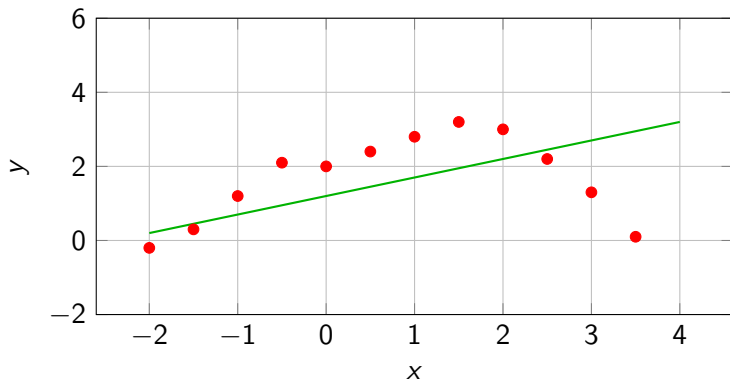
- Underfitting occurs when a model is too simple (high bias), failing to capture the underlying trend.

Overfitting vs Underfitting

- Underfitting occurs when a model is too simple (high bias), failing to capture the underlying trend.
 - High error on training and testing data.

Overfitting vs Underfitting

- Underfitting occurs when a model is too simple (high bias), failing to capture the underlying trend.
 - High error on training and testing data.



Overfitting vs Underfitting

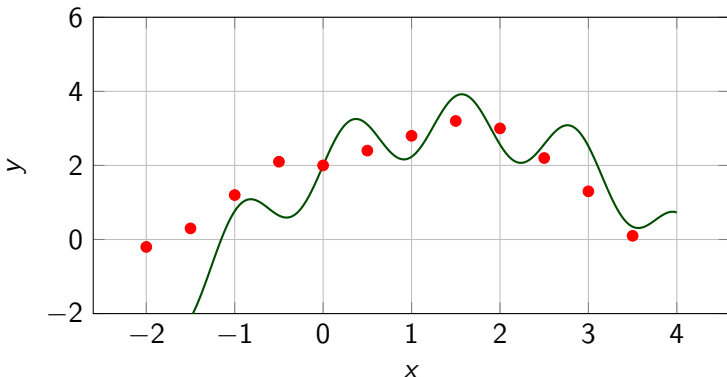
- Overfitting occurs when a model is too complex (high variance), capturing noise instead of the true underlying signal.

Overfitting vs Underfitting

- Overfitting occurs when a model is too complex (high variance), capturing noise instead of the true underlying signal.
 - Very low error on training data.
 - High error on testing data.

Overfitting vs Underfitting

- Overfitting occurs when a model is too complex (high variance), capturing noise instead of the true underlying signal.
 - Very low error on training data.
 - High error on testing data.

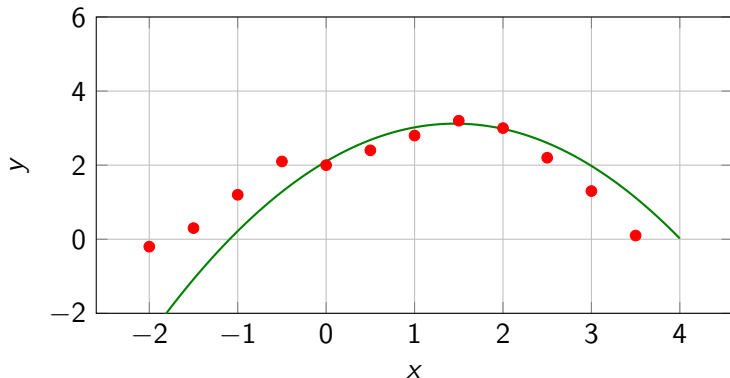


Overfitting vs Underfitting

- Bias-Variance Tradeoff: decreasing one type of error often increases the other. Regularization is a primary tool for navigating this tradeoff by introducing a small amount of bias to significantly reduce variance.

Overfitting vs Underfitting

- Bias-Variance Tradeoff: decreasing one type of error often increases the other. Regularization is a primary tool for navigating this tradeoff by introducing a small amount of bias to significantly reduce variance.



Vector Norm

Vector Norm

- A p -norm is a mathematical function that measures the magnitude or “length” of a vector in a vector space. The value of p (where $p \geq 1$) determines how the length is calculated.

Vector Norm

- A p -norm is a mathematical function that measures the magnitude or “length” of a vector in a vector space. The value of p (where $p \geq 1$) determines how the length is calculated.
- For a vector β with n elements, $\beta = [\beta_1, \beta_2, \dots, \beta_n]$, the p -norm is defined as:

$$\|\beta\|_p = \left(\sum_{i=1}^n |\beta_i|^p \right)^{1/p}$$

Vector Norm

- A p -norm is a mathematical function that measures the magnitude or “length” of a vector in a vector space. The value of p (where $p \geq 1$) determines how the length is calculated.
- For a vector β with n elements, $\beta = [\beta_1, \beta_2, \dots, \beta_n]$, the p -norm is defined as:

$$\|\beta\|_p = \left(\sum_{i=1}^n |\beta_i|^p \right)^{1/p}$$

- 1 Take the absolute value of each element in the vector.
- 2 Raise each absolute value to the power of p .
- 3 Sum all results.
- 4 Take p -th root of the sum.

Vector Norm

Different values of p give us different, commonly used norms.

Vector Norm

Different values of p give us different, commonly used norms.

- l_1 -norm: Also known as the Manhattan norm or taxicab norm. It's the sum of the absolute values of the elements.

$$\|\beta\|_1 = |\beta_1| + |\beta_2| + \cdots + |\beta_n|$$

Vector Norm

Different values of p give us different, commonly used norms.

- l_1 -norm: Also known as the Manhattan norm or taxicab norm. It's the sum of the absolute values of the elements.

$$\|\beta\|_1 = |\beta_1| + |\beta_2| + \cdots + |\beta_n|$$

- It encourages sparsity.
- By penalizing the sum of absolute values, the l_1 -norm finds it “cheaper” to push some coefficients all the way to zero rather than keeping them very small.
- Pushing a coefficient to zero is equivalent to removing that feature from the model entirely. This makes the l_1 -norm a powerful tool for automatic feature selection.

Vector Norm

Different values of p give us different, commonly used norms.

- l_2 -norm: This is the standard Euclidean norm, which corresponds to the intuitive, straight-line length of a vector.

$$\|\beta\|_2 = \sqrt{\beta_1^2 + \beta_2^2 + \cdots + \beta_n^2}$$

Vector Norm

Different values of p give us different, commonly used norms.

- l_2 -norm: This is the standard Euclidean norm, which corresponds to the intuitive, straight-line length of a vector.

$$\|\beta\|_2 = \sqrt{\beta_1^2 + \beta_2^2 + \cdots + \beta_n^2}$$

- It measures the straight-line distance from the origin.
- It shrinks all coefficients towards zero but does not force them to be exactly zero. This helps prevent overfitting by creating a simpler, less complex model that generalizes better to new data.
- It prefers smaller, more evenly distributed coefficients.

Vector Norm

Different values of p give us different, commonly used norms.

- l_∞ -norm: Also known as the maximum norm or Chebyshev distance. As p approaches infinity, the norm simplifies to the maximum absolute value of any element in the vector.

$$\|\beta\|_\infty = \max(|\beta_1|, |\beta_2|, \dots, |\beta_n|)$$

Vector Norm

Different values of p give us different, commonly used norms.

- l_∞ -norm: Also known as the maximum norm or Chebyshev distance. As p approaches infinity, the norm simplifies to the maximum absolute value of any element in the vector.

$$\|\beta\|_\infty = \max(|\beta_1|, |\beta_2|, \dots, |\beta_n|)$$

- The l_∞ -norm is used when the focus is on the single most extreme value rather than the overall magnitude.

Vector Norm

Different values of p give us different, commonly used norms.

- l_0 -norm: It is not technically a norm, but a function that counts the number of non-zero elements in a vector.

$$\|\beta\|_0 = \text{number of non-zero elements in } \beta$$

Vector Norm

Different values of p give us different, commonly used norms.

- l_0 -norm: It is not technically a norm, but a function that counts the number of non-zero elements in a vector.

$$\|\beta\|_0 = \text{number of non-zero elements in } \beta$$

- Finding the best subset of features is an NP-hard problem. It's computationally impossible to test every single combination of features for anything but the smallest datasets.
- The l_0 -norm is the theoretical ideal for creating a sparse model

Ridge Regression

Ridge Regression

- Ridge Regression modifies the standard linear regression cost function by adding a penalty for large coefficients.

$$\|y - X\beta\|_2^2 + \lambda\|\beta\|_2^2$$

Ridge Regression

- Ridge Regression modifies the standard linear regression cost function by adding a penalty for large coefficients.

$$\|y - X\beta\|_2^2 + \lambda\|\beta\|_2^2$$

- A larger λ means a stronger penalty, forcing the model to use smaller coefficients.
- The primary effect of the l_2 penalty is shrinking the model's coefficients toward zero.

Ridge Regression

- Ridge Regression modifies the standard linear regression cost function by adding a penalty for large coefficients.

$$\|y - X\beta\|_2^2 + \lambda\|\beta\|_2^2$$

- A larger λ means a stronger penalty, forcing the model to use smaller coefficients.
- The primary effect of the l_2 penalty is shrinking the model's coefficients toward zero.
 - It reduces the influence of each individual predictor.
 - It does not set any coefficients exactly to zero. All features are kept in the final model.
 - This is especially useful when features are highly correlated (multicollinearity), as it helps to stabilize the model.

Ridge Regression

Cost Function:

$$J(\beta) = (y - X\beta)^T (y - X\beta) + \lambda \beta^T \beta$$

Ridge Regression

Cost Function:

$$J(\beta) = (y - X\beta)^T (y - X\beta) + \lambda \beta^T \beta$$

To find the minimum, we compute the gradient of $J(\beta)$ with respect to β , denoted as $\frac{\partial J(\beta)}{\partial \beta}$.

Ridge Regression

Cost Function:

$$J(\beta) = (y - X\beta)^T (y - X\beta) + \lambda \beta^T \beta$$

To find the minimum, we compute the gradient of $J(\beta)$ with respect to β , denoted as $\frac{\partial J(\beta)}{\partial \beta}$.

The resulting gradient is:

$$\frac{\partial J(\beta)}{\partial \beta} = -2X^T y + 2(X^T X)\beta + 2\lambda \beta$$

Ridge Regression

Cost Function:

$$J(\beta) = (y - X\beta)^T (y - X\beta) + \lambda \beta^T \beta$$

To find the minimum, we compute the gradient of $J(\beta)$ with respect to β , denoted as $\frac{\partial J(\beta)}{\partial \beta}$.

The resulting gradient is:

$$\frac{\partial J(\beta)}{\partial \beta} = -2X^T y + 2(X^T X)\beta + 2\lambda \beta$$

Set the gradient to zero to find the optimal coefficients, $\hat{\beta}$:

$$-2X^T y + 2(X^T X)\hat{\beta} + 2\lambda \hat{\beta} = 0 \Rightarrow \hat{\beta} = (X^T X + \lambda I)^{-1} X^T y$$

LASSO Regression

LASSO Regression

- LASSO Regression (Least Absolute Shrinkage and Selection Operator) adds a penalty equal to the l_1 -norm of the coefficients.

$$\|y - X\beta\|_2^2 + \lambda\|\beta\|_1$$

LASSO Regression

- LASSO Regression (Least Absolute Shrinkage and Selection Operator) adds a penalty equal to the l_1 -norm of the coefficients.

$$\|y - X\beta\|_2^2 + \lambda\|\beta\|_1$$

- A larger λ means a stronger penalty, forcing the model to be sparse.
- The l_1 penalty forces the coefficients of the least important features to shrink all the way to exactly zero.

LASSO Regression

- LASSO Regression (Least Absolute Shrinkage and Selection Operator) adds a penalty equal to the l_1 -norm of the coefficients.

$$\|y - X\beta\|_2^2 + \lambda\|\beta\|_1$$

- A larger λ means a stronger penalty, forcing the model to be sparse.
- The l_1 penalty forces the coefficients of the least important features to shrink all the way to exactly zero.
 - If a feature's coefficient is zero, it's effectively removed from the model.
 - This creates a sparse model—a simpler model that only includes the most relevant predictors.
 - This makes LASSO a powerful tool for automatic feature selection.

LASSO Regression

Cost Function:

$$J(\beta) = (y - X\beta)^T (y - X\beta) + \lambda \|\beta\|_1$$

LASSO Regression

Cost Function:

$$J(\beta) = (y - X\beta)^T (y - X\beta) + \lambda \|\beta\|_1$$

- Unlike Ridge Regression, there is no simple, analytical formula (like the Normal Equation) for the LASSO solution.
- We cannot simply take the derivative of the cost function and set it to zero to solve for β .

LASSO Regression

Cost Function:

$$J(\beta) = (y - X\beta)^T (y - X\beta) + \lambda \|\beta\|_1$$

- Unlike Ridge Regression, there is no simple, analytical formula (like the Normal Equation) for the LASSO solution.
- We cannot simply take the derivative of the cost function and set it to zero to solve for β .
- The most common and intuitive algorithm for LASSO is **Coordinate Descent** and **Proximal Gradient Descent** (Lecture 1).

LASSO Regression

- For composite problems

$$\min_{\beta} F(\beta) = f(\beta) + g(\beta).$$

- $f(\beta)$: Convex, differentiable.
- $g(\beta)$: Convex, possibly non-differentiable.

LASSO Regression

- For composite problems

$$\min_{\beta} F(\beta) = f(\beta) + g(\beta).$$

- $f(\beta)$: Convex, differentiable.
 - $g(\beta)$: Convex, possibly non-differentiable.
- Update Rule:

$$\beta_{k+1} = \mathbf{prox}_{\alpha_k g}(\beta_k - \alpha_k \nabla f(\beta_k)).$$

LASSO Regression

- For composite problems

$$\min_{\beta} F(\beta) = f(\beta) + g(\beta).$$

- $f(\beta)$: Convex, differentiable.
- $g(\beta)$: Convex, possibly non-differentiable.
- Update Rule:

$$\beta_{k+1} = \mathbf{prox}_{\alpha_k g}(\beta_k - \alpha_k \nabla f(\beta_k)).$$

- Proximal Operator:

$$\mathbf{prox}_{\alpha g}(v) = \operatorname{argmin}_{\beta} \left(g(\beta) + \frac{1}{2\alpha} \|\beta - v\|_2^2 \right).$$

LASSO Regression

- For composite problems

$$\min_{\beta} F(\beta) = f(\beta) + g(\beta).$$

- $f(\beta)$: Convex, differentiable.
 - $g(\beta)$: Convex, possibly non-differentiable.
- Update Rule:

$$\beta_{k+1} = \mathbf{prox}_{\alpha_k g}(\beta_k - \alpha_k \nabla f(\beta_k)).$$

- Proximal Operator:

$$\mathbf{prox}_{\alpha g}(v) = \operatorname{argmin}_{\beta} \left(g(\beta) + \frac{1}{2\alpha} \|\beta - v\|_2^2 \right).$$

- For $g(\beta) = \lambda \|\beta\|_1$, **prox** is soft-thresholding:

$$\mathbf{prox}_{\alpha_k \lambda \|\cdot\|_1}(v_i) = \operatorname{sign}(v_i) \max(0, |v_i| - \lambda \alpha_k).$$

Comparison among Regular Regression, Ridge Regression and Lasso Regression

MATLAB Demonstration.

Summary of What You've Learned Today

- Linear Regression.
- Linear Regression with Regularization.
 - Vector Norm.
 - Ridge Regression.
 - LASSO Regression.

Summary of What You've Learned Today

- Linear Regression.
- Linear Regression with Regularization.
 - Vector Norm.
 - Ridge Regression.
 - LASSO Regression.

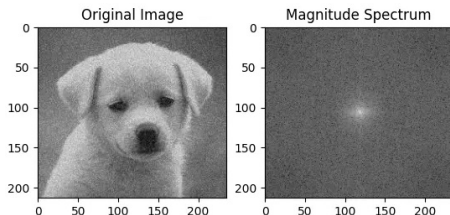
See you on Monday (7/14) for Lecture 6 on
Sparse Regression and Dynamical Systems.

Sparse Representation

- Many signals/images are compressible: most coefficients in some basis are nearly zero.

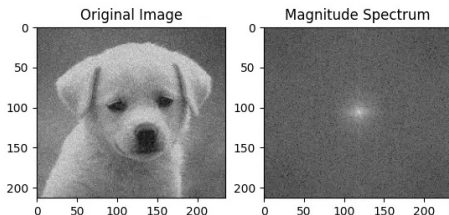
Sparse Representation

- Many signals/images are compressible: most coefficients in some basis are nearly zero.



Sparse Representation

- Many signals/images are compressible: most coefficients in some basis are nearly zero.



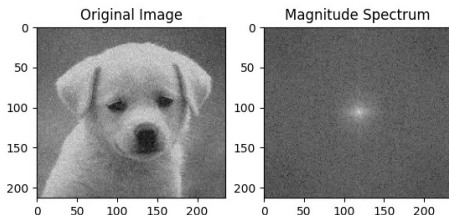
- Represent signal $\mathbf{x} \in \mathbb{R}^n$ as:

$$\mathbf{x} = \mathbf{A}\Psi\mathbf{s}$$

where $\mathbf{s}$ is a sparse representation, $\mathbf{A}$ is measurement matrix and Ψ is a known basis (e.g. Fourier, Principal Components).

Sparse Representation

- Many signals/images are compressible: most coefficients in some basis are nearly zero.



- Represent signal $\mathbf{x} \in \mathbb{R}^n$ as:

$$\mathbf{x} = \mathbf{A}\Psi\mathbf{s}$$

where $\mathbf{s}$ is a sparse representation, $\mathbf{A}$ is measurement matrix and Ψ is a known basis (e.g. Fourier, Principal Components).

- Goal:** Find a sparse vector $\mathbf{s}$ that represents the signal.

Compressed Sensing

Compressed Sensing

- Goal:
 - Reconstruct a high-dimensional signal from a small number of linear measurements.

Compressed Sensing

- Goal:
 - Reconstruct a high-dimensional signal from a small number of linear measurements.
- Challenge (Nyquist-Shannon sampling theorem):

Compressed Sensing

- Goal:
 - Reconstruct a high-dimensional signal from a small number of linear measurements.
- Challenge (Nyquist-Shannon sampling theorem):
 - To reconstruct a band-limited signal:

$$f_{\text{sampling}} > 2f_{\text{max}}$$

Compressed Sensing

- Goal:
 - Reconstruct a high-dimensional signal from a small number of linear measurements.
- Challenge (Nyquist-Shannon sampling theorem):
 - To reconstruct a band-limited signal:

$$f_{\text{sampling}} > 2f_{\text{max}}$$

- If signal contains frequencies up to 30 Hz $\Rightarrow$ must sample > 60 Hz.

Compressed Sensing

- Goal:
 - Reconstruct a high-dimensional signal from a small number of linear measurements.
- Challenge (Nyquist-Shannon sampling theorem):
 - To reconstruct a band-limited signal:

$$f_{\text{sampling}} > 2f_{\text{max}}$$

- If signal contains frequencies up to 30 Hz $\Rightarrow$ must sample > 60 Hz.
- Violation leads to aliasing — frequency folding.

Compressed Sensing

- Goal:
 - Reconstruct a high-dimensional signal from a small number of linear measurements.
- Challenge (Nyquist-Shannon sampling theorem):
 - To reconstruct a band-limited signal:

$$f_{\text{sampling}} > 2f_{\text{max}}$$

- If signal contains frequencies up to 30 Hz $\Rightarrow$ must sample > 60 Hz.
- Violation leads to aliasing — frequency folding.

But if the signal is sparse in frequency domain, we can beat the Nyquist-Shannon sampling theorem.

Compressed Sensing

- Assumption: Signal is sparse in some basis (e.g., Fourier). Only a few nonzero Fourier coefficients.

Compressed Sensing

- Assumption: Signal is sparse in some basis (e.g., Fourier). Only a few nonzero Fourier coefficients.
- Optimization problem:

$$\min \|\mathbf{x}\|_1 \quad \text{subject to} \quad \mathbf{y} = \mathbf{Ax}$$

Compressed Sensing

- Assumption: Signal is sparse in some basis (e.g., Fourier). Only a few nonzero Fourier coefficients.
- Optimization problem:

$$\min \|\mathbf{x}\|_1 \quad \text{subject to} \quad \mathbf{y} = \mathbf{A}\mathbf{x}$$

$$\min \|\mathbf{x}_{\text{freq}}\|_1 \quad \text{subject to} \quad \mathbf{y} = \mathbf{A}\mathbf{F}^\dagger \mathbf{x}_{\text{freq}}$$

Compressed Sensing

- Assumption: Signal is sparse in some basis (e.g., Fourier). Only a few nonzero Fourier coefficients.
- Optimization problem:

$$\min \|\mathbf{x}\|_1 \quad \text{subject to} \quad \mathbf{y} = \mathbf{A}\mathbf{x}$$

$$\min \|\mathbf{x}_{\text{freq}}\|_1 \quad \text{subject to} \quad \mathbf{y} = \mathbf{A}\mathbf{F}^\dagger \mathbf{x}_{\text{freq}}$$

- Compressed sensing allows exact reconstruction from far fewer samples and enables recovery from undersampled data.

Compressed Sensing

MATLAB Demonstration:

Compressed Sensing

MATLAB Demonstration:

- Signal length $n = 128$ with two frequencies.

Compressed Sensing

MATLAB Demonstration:

- Signal length $n = 128$ with two frequencies.
- Sparse in Fourier domain: $X[k] \neq 0$ for only a few k .

Compressed Sensing

MATLAB Demonstration:

- Signal length $n = 128$ with two frequencies.
- Sparse in Fourier domain: $X[k] \neq 0$ for only a few k .
- Sampled at $m = 40$ random time points.

Compressed Sensing

MATLAB Demonstration:

- Signal length $n = 128$ with two frequencies.
- Sparse in Fourier domain: $X[k] \neq 0$ for only a few k .
- Sampled at $m = 40$ random time points.
- Let $\mathbf{F}$ be the DFT matrix.
- Sparse coefficients: $\mathbf{x}_{\text{freq}}$.

Compressed Sensing

MATLAB Demonstration:

- Signal length $n = 128$ with two frequencies.
- Sparse in Fourier domain: $X[k] \neq 0$ for only a few k .
- Sampled at $m = 40$ random time points.
- Let $\mathbf{F}$ be the DFT matrix.
- Sparse coefficients: $\mathbf{x}_{\text{freq}}$.
- Observed samples:

$$\mathbf{y} = \mathbf{A} \cdot \mathcal{F}^{-1}\{\mathbf{x}_{\text{freq}}\} = \mathbf{A}\mathbf{F}^{\dagger}\mathbf{x}_{\text{freq}} = \mathbf{\Phi}\mathbf{x}_{\text{freq}}$$

Compressed Sensing

MATLAB Demonstration:

- Signal length $n = 128$ with two frequencies.
- Sparse in Fourier domain: $X[k] \neq 0$ for only a few k .
- Sampled at $m = 40$ random time points.
- Let $\mathbf{F}$ be the DFT matrix.
- Sparse coefficients: $\mathbf{x}_{\text{freq}}$.
- Observed samples:

$$\mathbf{y} = \mathbf{A} \cdot \mathcal{F}^{-1}\{\mathbf{x}_{\text{freq}}\} = \mathbf{A}\mathbf{F}^{\dagger}\mathbf{x}_{\text{freq}} = \mathbf{\Phi}\mathbf{x}_{\text{freq}}$$

- Recover frequency domain signal by solving:

$$\min \|\mathbf{x}_{\text{freq}}\|_1 \quad \text{subject to} \quad \mathbf{\Phi}\mathbf{x}_{\text{freq}} = \mathbf{y}$$

Robust Principal Component Analysis (RPCA)

Robust Principal Component Analysis (RPCA) is a modification of principal component analysis (PCA), which works well with respect to grossly corrupted observations.

Robust Principal Component Analysis (RPCA)

Robust Principal Component Analysis (RPCA) is a modification of principal component analysis (PCA), which works well with respect to grossly corrupted observations.

$$\min_{\mathbf{L}, \mathbf{S}} \|\mathbf{L}\|_* + \lambda \|\mathbf{S}\|_1 \quad \text{subject to} \quad \mathbf{D} = \mathbf{L} + \mathbf{S}$$

Robust Principal Component Analysis (RPCA)

Robust Principal Component Analysis (RPCA) is a modification of principal component analysis (PCA), which works well with respect to grossly corrupted observations.

$$\min_{\mathbf{L}, \mathbf{S}} \|\mathbf{L}\|_* + \lambda \|\mathbf{S}\|_1 \quad \text{subject to} \quad \mathbf{D} = \mathbf{L} + \mathbf{S}$$

- $\|\mathbf{L}\|_*$: nuclear norm (sum of singular values) $\rightarrow$ promotes low-rank
- $\|\mathbf{S}\|_1$: l_1 norm $\rightarrow$ promotes sparsity
- Solved via Inexact ALM (Augmented Lagrange Multiplier)

Robust Principal Component Analysis (RPCA)

Image Recovery via RPCA

Robust Principal Component Analysis (RPCA)

Image Recovery via RPCA

- Corrupted Image: consistent structure across images, sparse corruption.

Robust Principal Component Analysis (RPCA)

Image Recovery via RPCA

- Corrupted Image: consistent structure across images, sparse corruption.
- RPCA separates:

$$\mathbf{D} = \mathbf{L} + \mathbf{S}$$

- **D**: matrix of images (each column is one image)
- **L**: low-rank images (constant structure)
- **S**: sparse corruption (noise)